Université Paris Cité
Intelligence Artificielle Distribuée (IAD)

TrainMind

Système multi-agent pour la recommandation sportive argumentée

---

Rapport de projet

Conception d'un système multi-agent capable de produire une recommandation sportive personnalisée, argumentée et sécurisée.

Étudiant : MOHAMMEDI Noureddine
Formation : Intelligence Artificielle Distribuée (IAD)
Université Paris Cité
Enseignant : Professeur Pavlos Moraitis
Matière : Agents intelligents
Date : Mai 2026

Table des matières

Résumé **3**

1 Introduction et problématique **4**
1.1 Introduction . . . 4
1.2 Problématique . . . 4
1.3 Travail réalisé . . . 5
1.3.1 Architecture multi-agent JADE . . . 5
1.3.2 Deux modules de raisonnement rAison . . . 5
1.3.3 Négociation multi-tour . . . 5
1.3.4 Automate de négociation . . . 5
1.3.5 Interface console opérationnelle . . . 5

2 Fondements théoriques et modélisation **6**
2.1 Agent et système multi-agent . . . 6
2.2 Communication FIPA ACL . . . 6
2.3 Argumentation et LPP . . . 7
2.4 Modélisation GAIA . . . 7
2.4.1 Rôles . . . 7
2.4.2 Interactions principales . . . 8

3 Architecture et implémentation **9**
3.1 Organisation des fichiers . . . 9
3.2 Principe d'exécution . . . 9
3.3 Mémoire locale d'entraînement . . . 10
3.4 Comportements JADE . . . 11
3.5 Composants génériques et composants spécifiques . . . 11
3.6 Services JADE utilisés . . . 12
3.7 Modèle de données métier . . . 12
3.7.1 Choix de périmètre . . . 12
3.7.2 Variables d'entrée . . . 12
3.7.3 Sorties attendues . . . 13
3.8 Modules de raisonnement rAison . . . 13
3.8.1 Performance . . . 13
3.8.2 Recovery . . . 13
3.8.3 Principe d'orchestration : Java orchestre, rAison décide . . . 13
3.8.4 Exemples de règles Performance . . . 15
3.8.5 Exemples de règles Recovery . . . 15

4 Protocole de négociation **16**
4.1 Idée générale . . . 16
4.2 Contenu réellement échangé . . . 16
4.3 Pourquoi plusieurs tours ? . . . 18
4.4 Automate de négociation . . . 19
4.5 Durée de la négociation . . . 20
4.6 Lien avec les performatives FIPA . . . 20
4.7 Trace représentative . . . 21

5 Validation et démonstration **22**
5.1 Scénario 1 : course saine . . . 22
5.2 Scénario 2 : douleur articulaire critique . . . 22
5.3 Scénario 3 : surcharge ACWR . . . 22
5.4 Table de validation fonctionnelle . . . 23
5.5 Gestion des erreurs et robustesse . . . 23
5.6 Protocole de démonstration . . . 23
5.6.1 Cas nominal . . . 23
5.6.2 Cas à risque . . . 23
5.6.3 Cas de surcharge . . . 24
5.7 Exécution du projet . . . 24

6 Discussion, limites et perspectives **25**
6.1 Ce que le SMA apporte réellement . . . 25
6.2 Séparation entre orchestration et décision métier . . . 25
6.3 Limites . . . 25
6.4 Perspectives . . . 25

7 Conclusion **26**

Résumé

TrainMind est un système multi-agent destiné à produire une recommandation sportive quotidienne à partir de l'état de l'athlète : objectif, phase d'entraînement, sommeil, fatigue, douleur articulaire, HRV, charge ACWR et disponibilité. Le système ne fournit pas une décision isolée : il met en interaction deux agents experts, l'agent Performance, chargé de proposer une séance favorable à la progression, et l'agent Recovery, chargé d'évaluer le risque et d'imposer des garanties de sécurité.

Le projet articule JADE pour l'organisation multi-agent, un protocole de négociation inspiré de FIPA ACL et deux bases de décision rAison exprimées en Prolog/LPP. L'objectif principal est de montrer comment un programme sportif peut résulter d'une négociation argumentée entre agents, tout en maintenant une séparation explicite entre l'orchestration technique et les règles métier.

La version actuelle se concentre sur trois familles d'objectifs : course, force et hypertrophie. Les décisions métier sont confiées à rAison, tandis que Java assure l'orchestration des agents, des messages, de l'affichage utilisateur et de la boucle de négociation.

Chapitre 1
Introduction et problématique

1.1 Introduction

La recommandation sportive automatique constitue une problématique sensible. Une séance pertinente pour améliorer la performance peut devenir inadaptée si l'athlète dort insuffisamment, accumule une charge excessive ou signale une douleur articulaire. À l'inverse, une logique trop prudente risque de recommander systématiquement du repos et de réduire l'intérêt d'un programme d'entraînement personnalisé.

Le projet TrainMind modélise cette tension comme un problème multi-agent. Plutôt que de produire directement un résultat unique, le système met en dialogue deux points de vue complémentaires :

— **Performance** : proposer une séance cohérente avec l'objectif et le cycle d'entraînement ;

— **Recovery** : contrôler le risque, demander des adaptations et valider ou refuser la proposition.

La recommandation finale correspond ainsi à un compromis explicite : elle est construite, justifiée et ajustée au fil des échanges.

1.2 Problématique

La question centrale du projet est la suivante :

> Comment concevoir un système multi-agent capable de produire une recommandation sportive personnalisée, tout en rendant explicite la négociation entre performance et sécurité ?

Cette problématique impose plusieurs contraintes :

— séparer les responsabilités entre agents ;

— éviter que l'ensemble de la logique métier soit concentré dans Java ;

— garantir qu'un contexte utilisateur conduise à une réponse fiable ;

— rendre la négociation compréhensible pour l'utilisateur et justifiable dans un cadre académique ;

— conserver une architecture réutilisable, dans laquelle les modules de décision peuvent évoluer indépendamment du SMA.

1.3 Travail réalisé

Les contributions du projet peuvent être structurées autour de cinq éléments principaux.

1.3.1 Architecture multi-agent JADE

Le système est composé de quatre agents :

— `Agent_User`, qui collecte le contexte sportif et affiche le programme final ;

— `Agent_Manager`, qui coordonne la conversation et transmet la décision ;

— `Agent_P`, l'agent Performance ;

— `Agent_R`, l'agent Recovery.

Chaque agent délègue ses responsabilités à des comportements JADE distincts. Cette organisation limite la création d'agents monolithiques et rapproche le système d'une architecture SMA maintenable.

1.3.2 Deux modules de raisonnement rAison

Deux projets rAison sont mobilisés :

— **TrainMind_Performance** (`PRJ31575`) : choix de la séance, de la cible, du RPE, du volume et de la distance ;

— **TrainMind_Recovery** (`PRJ31625`) : évaluation du risque, contraintes de sécurité et validation du plan.

Les deux bases fonctionnent en mode Prolog. Les scénarios y formalisent les règles métier sous forme logique. Le code Java n'a pas pour rôle de remplacer ces règles : il les interroge et orchestre leurs résultats.

1.3.3 Négociation multi-tour

La négociation entre `Agent_P` et `Agent_R` suit une structure inspirée du Contract Net, enrichie par des contre-propositions de sécurité. L'agent Performance propose une séance ; l'agent Recovery en évalue le risque, puis demande les garanties nécessaires : volume réduit, RPE plafonné, distance validée, règle d'arrêt ou repos complet dans les cas critiques.

1.3.4 Automate de négociation

La logique de dialogue n'est pas implémentée comme une simple succession de conditions. Elle est regroupée dans un automate de négociation côté Java. Cet automate suit les contraintes effectivement retournées par rAison : il ne répète pas mécaniquement cinq tours, mais intègre les garanties déjà obtenues et détermine le moment où la proposition peut être acceptée.

1.3.5 Interface console opérationnelle

L'application est exécutable depuis le terminal. L'utilisateur peut répondre aux questions successivement ou saisir directement un contexte avancé sous forme `key=value`. La sortie finale présente la séance, les consignes, la justification synthétique et la trace de négociation.

Chapitre 2
Fondements théoriques et modélisation

2.1 Agent et système multi-agent

Un agent est une entité logicielle autonome située dans un environnement, capable de percevoir des informations, de raisonner et d'agir. Un système multi-agent regroupe plusieurs agents qui coopèrent, négocient ou se coordonnent pour résoudre un problème.

Dans TrainMind, l'apport du SMA ne se limite pas à la répartition du code. Il permet de représenter deux expertises en tension : l'amélioration de la performance et la préservation de l'intégrité physique.

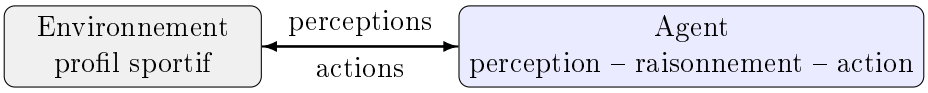


FIGURE 2.1 – Modèle classique agent–environnement, adapté au contexte TrainMind.

2.2 Communication FIPA ACL

FIPA ACL définit une famille de messages pour structurer les interactions entre agents. TrainMind utilise une sélection restreinte et lisible de performatives :

Performative	**Rôle dans TrainMind**
`inform`	transmettre un contexte, une contrainte appliquée ou un résultat final ;
`request`	demander à un agent de démarrer une tâche ;
`cfp`	ouvrir une négociation autour d'une proposition de plan ;
`propose`	proposer un plan ou une version corrigée ;
`reject-proposal`	refuser temporairement une proposition jugée incomplète ;
`query-if`	demander une garantie explicite ;
`accept-proposal`	accepter le plan final ;
`failure`	signaler l'impossibilité de produire une décision fiable.

Cette sélection suffit à rendre la conversation compréhensible, sans alourdir le projet par l'intégration exhaustive des performatives FIPA.

Exemple d'application

Dans TrainMind, `reject-proposal` est plus approprié que `refuse`. Si `Agent_P` propose une séance et que `Agent_R` l'évalue comme trop risquée, Recovery ne refuse pas d'exécuter sa tâche : il rejette uniquement la proposition actuelle. Il peut alors répondre `reject-proposal(unsafe_or_incomplete_plan)` puis demander une correction.
À l'inverse, `refuse` serait utilisé si un agent refusait d'exécuter une tâche demandée, par exemple si `Agent_P` ne pouvait pas lancer la négociation.

2.3 Argumentation et LPP

Le raisonnement rAison est utilisé comme une couche argumentative : plusieurs options peuvent être compatibles avec un contexte, puis les préférences et les mécanismes de résolution de conflit déterminent les options retenues. Cette approche convient au domaine sportif, car les décisions ne sont pas uniquement booléennes. Une séance peut être pertinente tout en nécessitant une adaptation, par exemple une réduction du volume ou un plafonnement du RPE.

2.4 Modélisation GAIA

La méthode GAIA aide à décrire le SMA à partir des rôles, permissions, responsabilités et protocoles.

2.4.1 Rôles

Rôle	**Responsabilités**	**Données manipulées**
Utilisateur	Fournir le contexte, recevoir le programme	Objectif, sommeil, fatigue, douleur, HRV, ACWR
Manager	Coordonner la session et transmettre les messages	Conversation, contexte, décision finale
Performance	Proposer une séance utile	Options Performance rAison, historique, volume, RPE
Recovery	Évaluer le risque et exiger des garanties	Options Recovery rAison, contraintes, validation

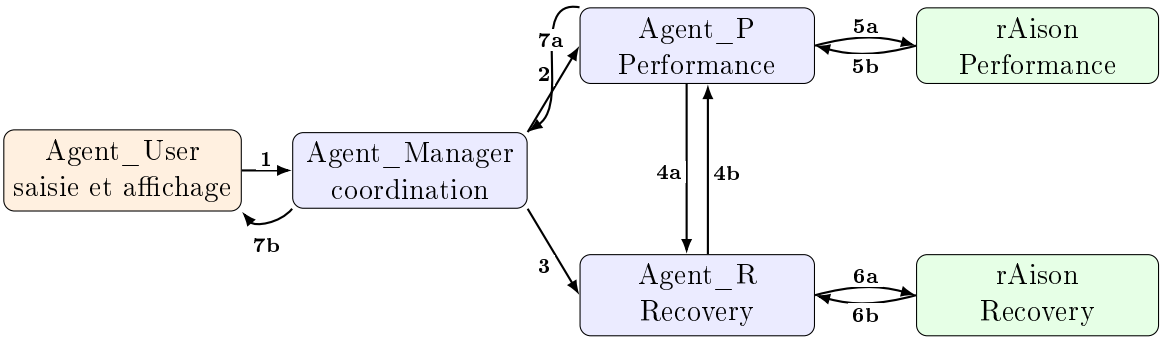


FIGURE 2.2 – Architecture générale du système multi-agent TrainMind.

2.4.2 Interactions principales

```
Agent_User    -> Agent_Manager : inform(context)
Agent_Manager -> Agent_P       : inform(context)
Agent_Manager -> Agent_R       : inform(context)
Agent_Manager -> Agent_P       : request(negotiate_plan)
Agent_P       -> Agent_R       : cfp(review_plan)
Agent_P       -> Agent_R       : propose(plan)
Agent_R       -> Agent_P       : reject-proposal / query-if / accept-proposal
Agent_P       -> Agent_Manager : inform(final_plan)
Agent_Manager -> Agent_User    : inform(final_plan)
```

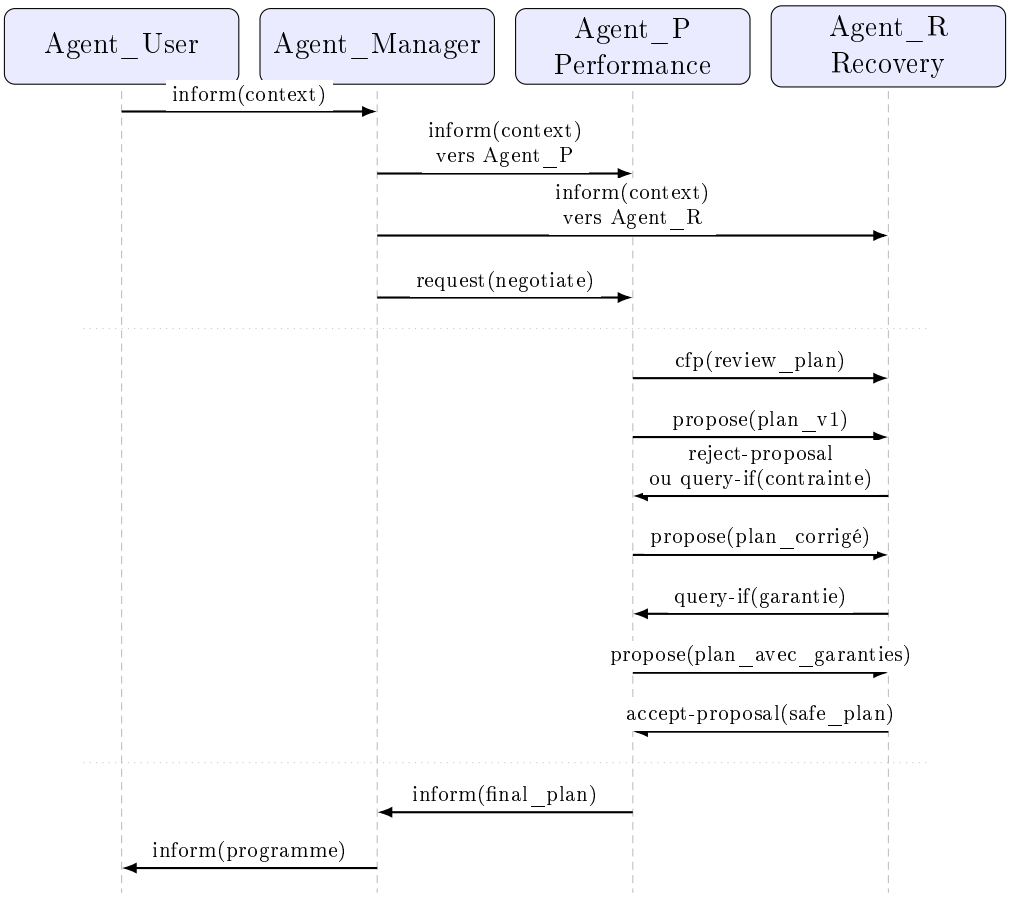


FIGURE 2.3 – Séquence simplifiée de négociation inspirée des protocoles FIPA ACL.

Chapitre 3
Architecture et implémentation

3.1 Organisation des fichiers

Le cœur du projet se trouve dans le dossier `TrainMind/`. Les agents JADE sont séparés des comportements et des clients rAison. Cette séparation facilite les évolutions : modifier un protocole de négociation ne nécessite pas de modifier la saisie utilisateur ou le client HTTP.

Fichier	**Rôle**
`Agent_User.java`	Agent d'interface console
`Agent_Manager.java`	Agent coordinateur
`Agent_P.java`	Agent Performance
`Agent_R.java`	Agent Recovery
`RaisonApiService.java`	Service d'accès à l'API rAison pour les deux projets
`RaisonFactMapper.java`	Transformation du contexte utilisateur en faits logiques
`TrainMindConfig.java`	Configuration des clés, identifiants de projets et paramètres d'exécution
`Argumentation ProtocolFSM.java`	Automate de négociation
`Negotiation DialogueFormatter.java`	Mise en forme lisible des échanges

3.2 Principe d'exécution

Le système suit le flux suivant :

1. l'utilisateur saisit son contexte ;
2. le manager diffuse le contexte aux agents ;
3. l'agent Performance interroge rAison Performance pour produire une proposition ;
4. l'agent Recovery interroge rAison Recovery pour analyser la proposition ;
5. les agents négocient jusqu'à acceptation ou échec ;
6. le manager transmet le programme final à l'utilisateur.

3.3 Mémoire locale d'entraînement

En complément des règles rAison, l'application conserve une mémoire locale dans le fichier `src/main/data/training_history.csv`. Ce fichier ne remplace pas le raisonnement logique : il fournit des données concrètes sur les séances précédentes.

Chaque ligne enregistrée contient :

date	target	category	exercise	load_kg	reps	rpe	distance_km

Après une séance de course, l'application enregistre la distance réalisée et le RPE. Après une séance de musculation ou d'hypertrophie, elle enregistre les exercices, les charges, les répétitions et le RPE. Cette mémoire permet à `Agent_P` de personnaliser les propositions suivantes : progression prudente des charges, validation des charges en musculation, ou cohérence des distances en course.

Le fichier CSV joue ainsi le rôle d'une mémoire minimale de l'athlète. rAison détermine les options retenues, tandis que Java utilise l'historique pour transformer ces options en programme concret.

3.4 Comportements JADE

La version finale ne concentre pas toute la logique dans les classes d'agents. Les comportements sont séparés dans des classes dédiées, ce qui correspond mieux à l'esprit de JADE : un agent est une entité autonome pouvant posséder plusieurs comportements.

Comportement	**Responsabilité**
`SaisirContexte UserBehaviour`	lit les réponses de l'utilisateur et construit le contexte sportif
`AfficherDecision UserBehaviour`	affiche le résultat final dans un langage compréhensible
`RecevoirContexte ManagerBehaviour`	reçoit le contexte utilisateur côté manager
`EnvoieContexte ParallelBehaviour`	transmet le contexte aux agents experts
`RecevoirReponses ArbitrageBehaviour`	récupère les réponses et gère l'arbitrage
`PerformanceNegotiation FSMBehaviour`	pilote l'agent Performance pendant la négociation
`RecoveryCfp FSMBehaviour`	pilote l'agent Recovery pendant l'évaluation

Cette organisation renforce la réutilisabilité du projet. Par exemple, l'interface console pourrait être remplacée par une interface graphique sans réécrire les modules de raisonnement Performance et Recovery.

3.5 Composants génériques et composants spécifiques

Un point important de l'architecture est la séparation entre les composants réutilisables et les composants propres au domaine sportif.

Composant	**Statut**	**Justification**
Agents JADE et comportements	Réutilisable	la structure de dialogue peut être reprise dans un autre domaine
Client rAison	Réutilisable	il encapsule les appels HTTP et le parsing des résultats
Historique de dialogue	Réutilisable	il permet de tracer les tours et d'éviter les échanges incohérents
Performatives ACL	Réutilisable	elles décrivent des intentions générales : proposer, refuser, accepter, informer
Scénarios rAison	Spécifique	ils encodent les règles sportives : ACWR, fatigue, sommeil, douleur
Options Performance et Recovery	Spécifique	elles correspondent au vocabulaire métier de TrainMind
Critères d'accord final	Hybride	la forme est générique, mais le contenu dépend du risque sportif

Cette distinction permet de présenter le projet comme une architecture d'agents intelligents, et non comme un programme de recommandation monolithique. Le même squelette pourrait être réutilisé dans un autre domaine où deux points de vue doivent négocier, à condition de remplacer les règles rAison et le vocabulaire métier.

3.6 Services JADE utilisés

JADE fournit également des services de plateforme qui structurent l'exécution :

— **AMS** (*Agent Management System*) : il gère le cycle de vie des agents sur la plateforme ;

— **DF** (*Directory Facilitator*) : il peut servir d'annuaire pour publier ou rechercher des services d'agents ;

— **ACLMessage** : il transporte les messages entre agents avec une performative, un contenu, une ontologie et un identifiant de conversation.

Dans TrainMind, l'utilisation la plus visible est l'échange d'`ACLMessage` entre les agents. L'identifiant de conversation permet de rattacher tous les messages d'une négociation au même contexte utilisateur.

3.7 Modèle de données métier

3.7.1 Choix de périmètre

Pour stabiliser l'application dans le cadre du prototype, le périmètre fonctionnel a été volontairement réduit à trois objectifs :

— **course** : entraînements running, endurance, tempo ou intervalles ;

— **force** : séances de musculation orientées force ;

— **masse** : séances d'hypertrophie.

L'objectif hybride a été retiré de l'interface afin d'éviter une ambiguïté fonctionnelle : un même contexte hybride pouvait raisonnablement conduire à une séance de course ou de musculation. Pour une démonstration académique, il est préférable de disposer d'un domaine mieux délimité et déterministe.

3.7.2 Variables d'entrée

Le contexte utilisateur est transformé en faits Prolog envoyés à rAison.

Variable	Type	Rôle
`goal(G)`	symbole	objectif principal : running, strength, hypertrophy
`cycle_phase(C)`	symbole	phase : accumulation, intensification, peaking, deload
`training_age(T)`	symbole	niveau : novice, intermediate, advanced
`sleep_hours(S)`	numérique	sommeil de la nuit précédente
`fatigue(F)`	numérique	fatigue subjective sur 10
`soreness(O)`	numérique	courbatures sur 10
`joint_pain(J)`	numérique	douleur articulaire sur 10
`hrv(H)`	symbole	état nerveux : low, normal, high
`acwr(A)`	numérique	ratio charge aiguë / charge chronique
`chronic_km(K)`	numérique	kilométrage chronique estimé
`risk_conservative_mode(RC)`	booléen	activation d'un mode prudent

3.7.3 Sorties attendues

`Agent_P` doit produire un programme défini par cinq dimensions :

— une séance ;

— une cible ;

— une intensité RPE ;

— un volume ;

— une distance.

`Agent_R` ne produit pas une séance concurrente. Il formule des contraintes : niveau de risque, restriction, validation ou repos obligatoire.

3.8 Modules de raisonnement rAison

3.8.1 Performance

Le projet Performance sélectionne une option dans chaque groupe logique :

— type de séance : course facile, sortie longue, tempo, intervalles, force, hypertrophie, mobilité, deload ;

— cible : running, recovery, push, pull, legs ;

— intensité : RPE rest, easy, moderate, hard, very hard ;

— volume : none, low, reduced, normal ;

— distance : none, short, medium, long.

Les scénarios couvrent notamment les cas suivants :

— course saine avec ACWR normal ;

— surcharge ACWR ;

— fatigue élevée, sommeil bas ou HRV bas ;

— douleur articulaire modérée ou critique ;

— force avancée et rotation push/pull/legs ;

— hypertrophie intermédiaire ;

— mode conservateur.

3.8.2 Recovery

Le projet Recovery produit des contraintes et validations :

— niveaux de risque : low, moderate, high, critical ;

— restrictions : reduce_volume, limit_rpe, low_intensity_only, complete_rest_required ;

— actions : récupération active, mobilité, sommeil prioritaire, repos complet ;

— garanties de négociation : distance validée, règle d'arrêt, plan validé.

Recovery assure ainsi le rôle de contrôleur de sécurité. Il ne remplace pas Performance, mais peut refuser une proposition ou demander des garanties complémentaires.

3.8.3 Principe d'orchestration : Java orchestre, rAison décide

Un choix central du projet consiste à éviter que Java devienne le moteur principal de la décision métier. Java interroge les deux bases rAison, puis applique uniquement les transformations nécessaires à la négociation : réduction du volume, plafonnement du RPE ou ajout d'une règle d'arrêt. Les décisions de haut niveau restent portées par les scénarios rAison.

Cette séparation rend le système plus justifiable : si une règle métier est contestée, elle peut être corrigée dans rAison plutôt que dans le code JADE.

3.8.4 Exemples de règles Performance

Branche	Conditions typiques	Décision
Course saine	ACWR normal, fatigue basse, douleur faible	running easy, long, tempo ou intervals
Surcharge ACWR	ACWR entre 1.3 et 1.5	deload, RPE easy, volume réduit
ACWR critique	ACWR supérieur à 1.5	mobilité, repos relatif, volume nul
Douleur critique	douleur articulaire au moins 7	cible récupération, RPE repos
Force avancée	objectif force, niveau avancé, douleur faible	strength session, cible push/pull/legs
Hypertrophie	objectif masse, niveau intermédiaire	hypertrophy session

Exemple d'application

Si l'utilisateur choisit `goal=strength` mais indique `joint_pain=8`, les scénarios de force sont bloqués par la condition `joint_pain(J), J < 7`. Le scénario de douleur critique devient prioritaire et produit une séance de récupération : `mobility_session`, `target_recovery`, `rpe_rest`, `volume_none`, `distance_none`.

3.8.5 Exemples de règles Recovery

Branche	Conditions typiques	Contraintes
Sommeil très bas	sommeil inférieur à 5.5 h	risk high, low intensity, limit RPE
Sommeil bas	sommeil entre 5.5 h et 6.5 h	risk moderate, reduce volume, limit RPE
Douleur critique	douleur articulaire au moins 7	complete rest required
ACWR haut	ACWR entre 1.3 et 1.5	risk high, reduce volume
ACWR critique	ACWR supérieur à 1.5	complete rest required
Proposition sûre	RPE et volume maîtrisés, contexte sain	plan validated

Exemple d'application

Si Performance propose une séance de course avec un volume trop élevé au premier tour, Recovery peut retourner `reduce_volume`. L'agent Performance conserve l'objectif général de la séance, mais réduit le volume, par exemple de 100% à 80%. La négociation ne remet pas en cause la proposition initiale : elle la rend plus sûre.

Chapitre 4
Protocole de négociation

4.1 Idée générale

La négociation ne relève pas d'un simple affichage illustratif. Elle permet de transformer une proposition initiale en programme plus sûr. Chaque tour apporte une adaptation concrète :

— réduction du volume ;

— plafonnement du RPE ;

— validation de la distance ;

— ajout d'une règle d'arrêt ;

— validation finale ou repos imposé.

Exemple d'application

Une garantie est une condition de sécurité demandée explicitement par Recovery. Par exemple, `query-if(stop_rule_defined)` signifie : "Peux-tu confirmer qu'une règle d'arrêt est prévue ?". Performance peut répondre `inform(stop_rule_defined)` en ajoutant : arrêt si douleur, crampe ou RPE trop élevé.

4.2 Contenu réellement échangé

À chaque proposition, Performance envoie à Recovery un objet de séance déjà structuré : type de séance, cible, RPE, volume, distance éventuelle et numéro de tour. Recovery n'évalue ainsi pas uniquement le contexte utilisateur initial ; il analyse également la proposition courante.

Les principaux faits transmis à Recovery sont :

— `proposal_round(RN)` : numéro du tour de négociation ;

— `proposal_rpe(R)` : intensité proposée ;

— `proposal_volume(V)` : volume proposé sous forme de pourcentage ;

— `proposal_running`, `proposal_strength` ou `proposal_recovery` : nature de la proposition.

Le volume n'est pas une valeur arbitraire calculée directement en Java. rAison Performance retourne une classe de volume, puis Java la traduit en pourcentage :

Option rAison	Interprétation Java
`volume_none`	0%
`volume_low`	30%
`volume_reduced`	70%
`volume_normal`	100%

La distance de course n'est pas une constante arbitraire. rAison choisit seulement une classe logique de distance : `distance_short`, `distance_medium` ou `distance_long`. Java transforme ensuite cette classe en kilomètres à partir de l'historique de l'athlète.

La formule utilisée est délibérément simple et explicable :

Élément	Rôle dans le calcul
`chronic_km`	base hebdomadaire habituelle, calculée depuis les 28 derniers jours
`last_run_km`	limite de progression par rapport à la dernière sortie
`acwr`	réduction si la charge récente est déjà élevée
`volume_percent`	conversion de la contrainte `reduce_volume` en distance
`available_minutes` et `vma_kmh`	vérification que la séance tient dans le temps disponible

En pratique, la classe rAison donne une part du kilométrage chronique :

Option rAison	Part de `chronic_km`	Intuition
`distance_short`	22%	fractionné ou séance courte
`distance_medium`	32%	footing ou tempo modéré
`distance_long`	45%	sortie longue

La distance est ensuite bornée par plusieurs règles de sécurité :

— progression maximale depuis la dernière sortie : environ +15% pour une séance normale, +20% pour une sortie longue, +5% pour des intervalles ;

— réduction si l'ACWR dépasse 1.0, et réduction forte si l'ACWR est entre 1.3 et 1.5 ;

— plafond selon le temps disponible et la VMA ;

— plafond par type de séance : intervalles plus courts que tempo, tempo plus court que sortie longue.

Si l'historique est encore peu renseigné, l'application évite de sous-estimer artificiellement le niveau de l'athlète. Elle utilise alors la dernière sortie connue comme garde-fou minimal : une dernière sortie longue fournit une estimation minimale du kilométrage chronique compatible avec cette sortie. Cette règle empêche un historique incomplet de transformer toutes les séances en distances excessivement faibles.

Ainsi, `running_long` ne correspond pas à une distance fixe, par exemple 10 km, et ne peut évidemment pas conduire à une distance extrême comme 100 km. Il signifie : "choisir une sortie longue", puis calculer une distance compatible avec l'historique réel de l'athlète. Par exemple, si le kilométrage chronique est de 22 km/semaine, `distance_long` vise environ `22 * 0.45 = 9.9` km avant l'application des plafonds. Si Recovery impose `reduce_volume`, le volume passe par exemple de 100% à 80%, et cette distance devient environ `9.9 * 0.8 = 7.9` km.

Lorsqu'une contrainte `reduce_volume` est demandée, Performance ne modifie pas seulement un libellé d'affichage. Il reconstruit le programme :

— en course, la distance calculée depuis l'historique est ajustée proportionnellement : une sortie estimée à 9.9 km devient environ 7.9 km à 80% ;

— en force ou en hypertrophie, le nombre de séries est réduit selon le même pourcentage : par exemple 4 séries deviennent 3 séries à 80% ;

— si Recovery demande `limit_rpe`, le RPE de la proposition est plafonné et les exercices sont recalculés avec cette intensité ;

— si Recovery demande `stop_rule_defined`, la règle d'arrêt est ajoutée aux consignes finales.

Contrainte Recovery	Cas typique	Effet sur la proposition Performance
`reduce_volume`	fatigue, ACWR élevé, volume trop haut	baisse le volume ; en course la distance diminue, en musculation le nombre de séries diminue

`limit_rpe`	sommeil bas, RPE trop haut, HRV fragile	plafonne l'intensité ; le RPE du plan est recalculé à une valeur plus sûre
`low_intensity_only`	fatigue haute, HRV bas, risque élevé	force une séance légère et bloque les intensités dures
`avoid_pain_zone`	douleur articulaire modérée	impose d'éviter la zone douloureuse dans le plan final
`distance_validated`	séance de course	confirme que la distance est cohérente avec le volume et l'historique
`load_validated`	force ou hypertrophie	confirme que les charges sont calculées depuis l'historique et restent conservatrices
`rest_times_defined`	force, puissance, hypertrophie	ajoute les temps de repos adaptés entre les séries
`stop_rule_defined`	tout plan accepté	ajoute une règle d'arrêt : douleur, crampe, technique dégradée ou RPE réel trop haut
`complete_rest_required`	douleur sévère ou ACWR critique	arrête la négociation sportive et impose repos/récupération

Exemple d'application

Cas running. Performance propose `running_long`, `rpe_moderate`, `volume_normal`, `distance_long`. Avec un historique de 22 km par semaine, Java calcule environ 9.9 km, RPE 6, volume 100%. Si Recovery demande `reduce_volume`, la proposition suivante devient environ 7.9 km, volume 80%, RPE 6.

Cas hypertrophie. Performance propose une séance de masse avec 4 séries de 10 répétitions sur l'exercice principal. Si Recovery demande une réduction de volume, la proposition suivante conserve le même objectif, mais passe à environ 3 séries au lieu de 4. La négociation adapte ainsi le programme sans modifier inutilement l'objectif sportif.

Une limite assumée de la version actuelle est que Recovery raisonne sur `proposal_volume(V)` et `proposal_rpe(R)`, mais ne reçoit pas encore explicitement le fait `proposal_distance_km(D)`. La distance est toutefois recalculée côté Performance et visible dans les propositions successives. Une amélioration future consisterait à ajouter ce fait dans rAison Recovery afin de permettre une règle directe sur la distance en kilomètres.

Enfin, l'automate Java protège le dialogue contre un cas pratique : si Recovery ne trouve aucun scénario rAison spécifique alors que le contexte est sain, il ne transforme pas la séance en repos complet. Il demande plutôt une garantie minimale cohérente avec le type de proposition : réduction du volume, validation de charge, temps de repos ou règle d'arrêt. En revanche, lorsque le contexte est réellement critique, le repli de sécurité demeure strict.

4.3 Pourquoi plusieurs tours ?

Un seul échange pourrait suffire à obtenir une décision technique, mais il ne permettrait pas d'illustrer une véritable négociation entre agents. Le projet impose par conséquent une discussion argumentée, accompagnée d'un minimum de garanties formelles. Cette structure rend visible le rôle de chaque agent :

— Performance défend l'intérêt sportif de la séance ;

— Recovery demande des concessions lorsque le risque est non nul ;

— l'accord final n'est obtenu qu'après intégration des garanties.

4.4 Automate de négociation

L'automate suit les contraintes retournées par rAison Recovery. Son comportement peut être résumé ainsi :

1. si Recovery retourne `complete_rest_required`, la négociation s'arrête ;
2. si Recovery retourne `reduce_volume`, Performance réduit le volume ;
3. si Recovery retourne `limit_rpe`, Performance plafonne le RPE ;
4. si Recovery demande une validation de distance ou une règle d'arrêt, Performance les ajoute ;
5. si `plan_validated` est obtenu avec les garanties minimales, le plan est accepté.

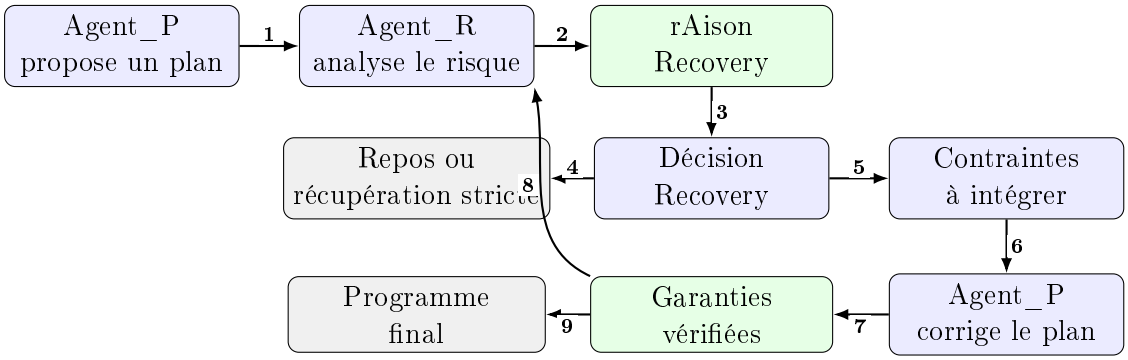


FIGURE 4.1 – Automate simplifié de négociation entre Performance et Recovery.

4.5 Durée de la négociation

La négociation n'a pas vocation à être identique pour tous les contextes. Le nombre de tours dépend du niveau de risque et des garanties demandées.

— **Cas critique** : Recovery peut imposer immédiatement le repos ou la récupération stricte.

— **Cas sain** : plusieurs tours permettent de formaliser les garanties, même si le risque est faible.

— **Cas intermédiaire** : Recovery demande une ou plusieurs adaptations avant validation.

Pour la démonstration, l'application met en avant une négociation visible, généralement composée de plusieurs échanges. Ce choix est volontaire : il permet de montrer le rôle des agents et d'éviter de réduire le projet à un simple appel API.

4.6 Lien avec les performatives FIPA

Les messages affichés dans la trace gardent une forme courte, mais ils correspondent à une intention FIPA claire :

Message	Interprétation
`propose (training_plan)`	Performance propose un programme initial
`reject-proposal (...)`	Recovery refuse temporairement car une garantie manque
`query-if(...)`	Recovery demande explicitement une preuve ou une condition
`inform(...)`	Performance informe qu'une contrainte a été appliquée
`accept-proposal (...)`	Recovery accepte le plan final

L'objectif n'est pas de reproduire mécaniquement l'ensemble des protocoles FIPA, mais d'utiliser les performatives adaptées au dialogue métier.

4.7 Trace représentative

La trace suivante illustre un cas de course sans facteur de risque majeur. Les formulations sont volontairement concises afin que la négociation reste lisible pendant la démonstration.

```
Round 1
Agent_P [propose] : Plan initial : sortie longue, 9.9 km, RPE 6, volume 100%.
Agent_R [reject-proposal] : Risque modere. Demande : volume -20%.

Round 2
Agent_P [propose] : Nouveau plan : sortie longue, 7.9 km, RPE 6, volume 80%.
Agent_R [query-if] : Ajouter une distance validee.

Round 3
Agent_P [propose] : Nouveau plan : distance validee, 7.9 km, RPE 6.
Agent_R [query-if] : Ajouter une regle d'arret.

Round 4
Agent_P [propose] : Nouveau plan : regle d'arret ajoutee.
Agent_R [accept-proposal] : Plan valide.
```

Selon le contexte, la négociation peut être plus courte ou plus stricte. En cas de douleur critique ou d'ACWR critique, Recovery peut imposer directement une réponse de sécurité. Dans les cas favorables, le système conserve plusieurs échanges afin de montrer l'intégration progressive des garanties.

Chapitre 5
Validation et démonstration

5.1 Scénario 1 : course saine

Contexte :

```
goal=running, cycle_phase=accumulation, training_age=intermediate,
sleep_hours=7.5, fatigue=4, soreness=3, joint_pain=0,
hrv=normal, acwr=1.0, chronic_km=22, available_minutes=60
```

Résultat attendu : une séance de course en endurance, avec RPE modéré, distance calculée et volume adapté. La négociation sert surtout à formaliser les garanties.

```
Round 1 : Performance propose une sortie running.
Round 2 : Recovery demande une reduction du volume.
Round 3 : Performance valide la distance et plafonne le RPE.
Round 4 : Recovery demande une regle d'arret explicite.
Round 5 : Recovery accepte le plan.
```

5.2 Scénario 2 : douleur articulaire critique

Contexte :

```
goal=strength, training_age=advanced, fatigue=5,
joint_pain=8, joint_pain_zone=knee, acwr=1.0
```

Résultat attendu : les scénarios de force sont bloqués par `joint_pain < 7`. Le scénario de douleur critique devient prioritaire et produit une séance de récupération ou de mobilité très prudente, avec `rpe_rest`, `volume_none` et `target_recovery`. Ce cas montre que la sécurité prime sur la performance.

5.3 Scénario 3 : surcharge ACWR

Contexte :

```
goal=running, acwr=1.5, fatigue=0, sleep_hours=8,
joint_pain=0, hrv=high
```

Résultat attendu : le système ne doit pas produire une séance intense malgré des indicateurs favorables apparents. La charge récente est élevée ; Performance bascule ainsi sur un scénario de sécurité : `deload_session`, `rpe_easy`, `volume_reduced`, `target_recovery`.

5.4 Table de validation fonctionnelle

Test	Contexte	Résultat attendu
Course saine	running, accumulation, fatigue 4, ACWR 1.0, douleur 0	course modérée, négociation avec garanties
Course en intensification	running, intensification, advanced, ACWR 1.0, fatigue 4	intervalles ou tempo selon niveau
Surcharge ACWR	running, ACWR 1.5, fatigue basse	deload, volume réduit
Douleur critique	strength, joint pain 8	mobilité/repos, aucune séance force
Sommeil très bas	running, sleep 5.0	séance très prudente ou récupération
Force saine	strength, advanced, douleur faible	force push/pull/legs selon historique

5.5 Gestion des erreurs et robustesse

Un problème important rencontré pendant le projet concernait l'absence de réponse rAison pour certains contextes. Ce comportement est problématique : si aucun scénario ne couvre le contexte, Java ne doit pas inventer une décision.

La règle retenue est donc la suivante :

> Si rAison ne retourne aucune solution fiable, Java signale une décision impossible au lieu de produire une séance arbitraire.

Ce choix a permis d'identifier plusieurs lacunes de couverture dans les bases de décision : ACWR élevé, douleur articulaire, mode conservateur et proposition Recovery saine au premier tour. Ces lacunes ont ensuite été corrigées dans rAison. Ces corrections illustrent un principe important du projet : renforcer la couche de raisonnement logique plutôt que compenser ses manques par des contournements dans l'orchestration Java.

5.6 Protocole de démonstration

La démonstration du prototype repose sur trois cas représentatifs. Ils ne constituent pas une validation médicale, mais permettent de vérifier que l'architecture SMA applique correctement les règles de performance, de récupération et de négociation.

5.6.1 Cas nominal

L'application est lancée avec `./run.sh` et un contexte de course sans facteur de risque majeur. Le résultat attendu est une séance lisible, négociée et accompagnée de garanties explicites.

5.6.2 Cas à risque

Un contexte avec douleur articulaire élevée, par exemple `joint_pain=8`, vérifie que la sécurité domine la performance. La séance de force doit être bloquée ou remplacée par une réponse prudente.

5.6.3 Cas de surcharge

Un contexte avec ACWR élevé, par exemple `acwr=1.5`, vérifie que le système réduit automatiquement la charge même lorsque les autres indicateurs semblent favorables.

Ces trois cas couvrent les comportements essentiels attendus : produire une proposition pertinente, la corriger en présence de risque, puis fournir une recommandation finale argumentée.

5.7 Exécution du projet

Le script `run.sh` compile automatiquement tous les fichiers Java situés sous `src/main/java/`, puis lance l'application.

```
cd TrainMind
./run.sh
```

Le fichier `.env` contient la clé API rAison et les identifiants des deux projets. Il n'est pas versionné.

Chapitre 6
Discussion, limites et perspectives

6.1 Ce que le SMA apporte réellement

Le SMA rend explicites les responsabilités. Dans une application classique, les règles de sécurité seraient souvent mêlées aux règles de performance. Ici, le conflit est représenté par deux agents distincts. Cette modélisation permet de rendre visible la progression du raisonnement : proposition, critique, adaptation, validation.

6.2 Séparation entre orchestration et décision métier

Implémenter toutes les règles en Java aurait simplifié l'intégration à court terme, mais aurait réduit l'intérêt du système multi-agent. La décision métier aurait été dispersée dans des conditions impératives, moins lisibles et plus difficiles à justifier. Avec rAison, les scénarios restent inspectables et modifiables sans nécessiter la réécriture de l'orchestration JADE.

6.3 Limites

Le système reste un prototype académique. Les limites principales sont :

— les données physiologiques sont déclaratives et non issues de capteurs réels ;

— la qualité du résultat dépend fortement de la couverture des scénarios rAison ;

— le système n'utilise pas encore de modèle probabiliste ou d'apprentissage automatique ;

— la négociation est structurée, mais reste guidée par des règles déterministes ;

— la validation médicale réelle est hors périmètre.

6.4 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées :

— ajouter des scénarios rAison pour des cas plus fins : compétition, retour de blessure, fatigue chronique ;

— connecter l'historique d'entraînement à un calcul ACWR plus complet ;

— enrichir Recovery avec davantage d'actions : cardio très léger, mobilité ciblée, sommeil prioritaire ;

— produire une interface graphique ;

— ajouter une suite de tests automatisés avec des contextes représentatifs ;

— exporter la trace de négociation pour analyse.

Chapitre 7
Conclusion

TrainMind montre comment un système multi-agent peut produire une recommandation sportive argumentée en séparant clairement les responsabilités. L'agent Performance propose une séance cohérente avec l'objectif d'entraînement, l'agent Recovery évalue les risques et impose des garanties, tandis que l'agent Manager coordonne l'échange. La décision finale ne se réduit donc pas à un calcul isolé : elle résulte d'une interaction structurée entre plusieurs expertises complémentaires.

L'apport principal du projet est de rendre visible le compromis entre progression et sécurité. Une proposition peut être critiquée, adaptée puis acceptée lorsque les contraintes de récupération sont satisfaites. À l'inverse, un contexte à risque, tel qu'une douleur articulaire importante ou une charge ACWR élevée, conduit à une recommandation plus prudente. Cette logique justifie l'utilisation d'un SMA plutôt qu'une architecture monolithique.

Sur le plan technique, JADE fournit la plateforme d'agents et les échanges inspirés de FIPA ACL, tandis que rAison porte les règles de décision métier, exprimées sous forme logique/Prolog. Cette séparation renforce l'explicabilité du système : les règles sportives peuvent évoluer sans modifier l'orchestration Java, et la trace de négociation permet de comprendre les concessions successives.

La version actuelle reste un prototype académique centré sur la course, la force et l'hypertrophie. Elle constitue néanmoins une base cohérente pour étendre le système vers d'autres objectifs sportifs, enrichir les scénarios de récupération et automatiser davantage la validation des contextes limites.